
Carrito de Compras Persistente (Riverpod + Isar)

1. Configuración de Dependencias ([pubspec.yaml](#))

Asegúrate de incluir estas versiones en tu archivo de configuración:

```
dependencies:
  flutter:
    sdk: flutter
  flutter_riverpod: ^2.5.1
  riverpod_annotation: ^2.3.4
  isar: ^3.1.0+1
  isar_flutter_libs: ^3.1.0+1
  path_provider: ^2.1.2

dev_dependencies:
  riverpod_generator: ^2.3.9
  isar_generator: ^3.1.0+1
  build_runner: ^2.4.8
```

2. Modelos de Datos ([lib/models/](#))

producto.dart

```
import 'package:isar/isar.dart';
part 'producto.g.dart';

@collection
class Producto {
  Id id = Isar.autoIncrement;
  @Index(unique: true)
  late String nombre;
  late double precio;
  late String categoria;
}
```

carrito_item.dart

```
import 'package:isar/isar.dart';
import 'producto.dart';
part 'carrito_item.g.dart';

@collection
class CarritoItem {
```

```
Id id = Isar.autoIncrement;
final producto = IsarLink<Producto>();
late int cantidad;
}
```

3. Servicio de Base de Datos (`lib/services/db_service.dart`)

```
import 'package:isar/isar.dart';
import 'package:path_provider/path_provider.dart';
import '../models/producto.dart';
import '../models/carrito_item.dart';

class DBService {
  static late Isar isar;

  static Future<void> init() async {
    final dir = await getApplicationDocumentsDirectory();
    isar = await Isar.open(
      [ProductoSchema, CarritoItemSchema],
      directory: dir.path,
    );
    await _seedDatabase();
  }

  static Future<void> _seedDatabase() async {
    final count = await isar.productos.count();
    if (count > 0) return;

    final catalogo = [
      Producto()..nombre = 'Monitor 4K'..precio = 350.0..categoria = 'Hardware',
      Producto()..nombre = 'Teclado Mecánico'..precio = 85.0..categoria =
'Periféricos',
      Producto()..nombre = 'Mouse Gamer'..precio = 45.0..categoria =
'Periféricos',
      Producto()..nombre = 'Laptop i7'..precio = 1200.0..categoria = 'Equipos',
    ];

    await isar.writeTxn(() async => await isar.productos.putAll(catalogo));
  }
}
```

4. Gestión de Estado (`lib/providers/`)

catalogo_provider.dart

```
import 'package:riverpod_annotation/riverpod_annotation.dart';
import '../models/producto.dart';
```

```
import '../services/db_service.dart';
import 'package:isar/isar.dart';

part 'catalogo_provider.g.dart';

@riverpod
class Catalogo extends _$Catalogo {
  @override
  Future<List<Producto>> build() async => await
    DBService.isar.productos.where().findAll();
}
```

carrito_provider.dart

```
import 'package:riverpod_annotation/riverpod_annotation.dart';
import 'package:isar/isar.dart';
import '../models/producto.dart';
import '../models/carrito_item.dart';
import '../services/db_service.dart';

part 'carrito_provider.g.dart';

@riverpod
class Carrito extends _$Carrito {
  @override
  Future<List<CarritoItem>> build() async {
    final items = await DBService.isar.carritoItems.where().findAll();
    for (var item in items) { await item.producto.load(); }
    return items;
  }

  Future<void> agregarProducto(Producto producto) async {
    await DBService.isar.writeTxn(() async {
      final existente = await DBService.isar.carritoItems
        .filter().producto((q) => q.idEqualTo(producto.id)).findFirst();

      if (existente != null) {
        existente.cantidad++;
        await DBService.isar.carritoItems.put(existente);
      } else {
        final nuevoItem = CarritoItem()..cantidad = 1;
        nuevoItem.producto.value = producto;
        await DBService.isar.carritoItems.put(nuevoItem);
        await nuevoItem.producto.save();
      }
    });
    ref.invalidateSelf();
  }

  Future<void> eliminarItem(int id) async {
    await DBService.isar.writeTxn(() async => await
```

```

DBService.isar.carritoItems.delete(id));
  ref.invalidateSelf();
}

Future<void> vaciarCarrito() async {
  await DBService.isar.writeTxn(() async => await
DBService.isar.carritoItems.clear());
  ref.invalidateSelf();
}
}

@riverpod
double carritoTotal(Ref ref) {
  final carritoAsync = ref.watch(carritoProvider);
  return carritoAsync.maybeWhen(
    data: (items) => items.fold(0, (sum, item) => sum +
(item.producto.value!.precio * item.cantidad)),
    orElse: () => 0.0,
  );
}

```

5. Interfaz de Usuario (`lib/screens/tienda_screen.dart`)

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../providers/catalogo_provider.dart';
import '../providers/carrito_provider.dart';

class TiendaScreen extends ConsumerWidget {
  const TiendaScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final catalogoAsync = ref.watch(catalogoProvider);
    final carritoAsync = ref.watch(carritoProvider);
    final total = ref.watch(carritoTotalProvider);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Ingeniería Store'),
        actions: [
          Center(child: Padding(
            padding: const EdgeInsets.only(right: 16),
            child: Text('Total: \${total.toStringAsFixed(2)}', style: const
TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
          )),
        ],
      ),
      body: Column(
        children: [

```

```

    _SectionTitle(title: 'Catálogo de Productos'),
    Expanded(
      flex: 2,
      child: catalogoAsync.when(
        data: (prods) => ListView.builder(
          itemCount: prods.length,
          itemBuilder: (_, i) => ListTile(
            title: Text(prods[i].nombre),
            subtitle: Text('\${prods[i].precio}'),
            trailing: IconButton(icon: const Icon(Icons.add_shopping_cart,
color: Colors.green),
              onPressed: () =>
ref.read(carritoProvider.notifier).agregarProducto(prods[i])),
          ),
        ),
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => Center(child: Text('Error: $e')),
      ),
    ),
    const Divider(),
    _CartHeader(onClear: () =>
ref.read(carritoProvider.notifier).vaciarCarrito()),
    Expanded(
      flex: 1,
      child: carritoAsync.when(
        data: (items) => items.isEmpty
          ? const Center(child: Text('Carrito vacío'))
          : ListView.builder(
              itemCount: items.length,
              itemBuilder: (_, i) => ListTile(
                leading: CircleAvatar(child: Text('${items[i].cantidad}')),
                title: Text(items[i].producto.value?.nombre ?? '...'),
                trailing: IconButton(icon: const Icon(Icons.delete, color:
Colors.red),
                  onPressed: () =>
ref.read(carritoProvider.notifier).eliminarItem(items[i].id)),
              ),
            ),
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => const SizedBox(),
      ),
    ),
  ],
),
);
}
}

```

```

class _SectionTitle extends StatelessWidget {
  final String title;
  const _SectionTitle({required this.title});
  @override
  Widget build(BuildContext context) => Padding(padding: const EdgeInsets.all(8),
    child: Text(title, style: const TextStyle(fontSize: 18, fontWeight:

```

```

FontWeight.bold)));
}

class _CartHeader extends StatelessWidget {
  final VoidCallback onClear;
  const _CartHeader({required this.onClear});
  @override
  Widget build(BuildContext context) => Padding(
    padding: const EdgeInsets.symmetric(horizontal: 16),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        const Text('Tu Carrito', style: TextStyle(fontSize: 18, fontWeight:
FontWeight.bold)),
        TextButton.icon(onPressed: onClear, icon: const Icon(Icons.delete_sweep,
color: Colors.red), label: const Text('Vaciar', style: TextStyle(color:
Colors.red)))
      ],
    ),
  );
}

```

6. Punto de Entrada (`lib/main.dart`)

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'services/db_service.dart';
import 'screens/tienda_screen.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await DBService.init();
  runApp(const ProviderScope(child: CarritoApp()));
}

class CarritoApp extends StatelessWidget {
  const CarritoApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorScheme: ColorScheme.fromSeed(seedColor:
Colors.indigo), useMaterial3: true),
      home: const TiendaScreen(),
    );
  }
}

```

Recordatorio final:

No olvides ejecutar en la terminal: `dart run build_runner build` Esto generará los archivos `.g.dart` necesarios para Isar y Riverpod.